

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Модели данных и системы управления базами данных

К защите допустить:

И.О. Заведующего кафедрой
информатики

_____ С. И. Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

АЭРОПОРТ

БГУИР КП 1-40 04 01 009 ПЗ

Студент

Е. С. Кахновский

Руководитель

В. С. Плинка

Минск 2024

СОДЕРЖАНИЕ

Введение.....	5
1 Обзор существующих аналогов.....	6
1.1 Aviasales	6
1.2 Skyscanner	8
1.3 FlightAware	9
2 Функциональные требования и инструменты разработки	11
2.1 Функциональные требования к системе.....	11
2.2 Выбор СУБД.....	12
3 Проектирование базы данных.....	20
3.1 Инфологическая модель.....	20
3.2 Даталогическая модель	22
3.3 Модель миграции данных	25
4 Разработка базы данных	26
4.1 Физическая модель базы данных	26
4.2 Создание таблиц.....	27
4.3 Оптимизация производительности	29
4.4 Триггеры и процедуры	30
4.5 Запросы	31
5 Тестирование работоспособности базы данных	33
Заключение	35
Список литературных источников	36
ПРИЛОЖЕНИЕ А (обязательное) Листинг кода.....	37
ПРИЛОЖЕНИЕ Б (обязательное) Схема базы данных.....	45
ПРИЛОЖЕНИЕ В (обязательное) Ведомость документов	46

ВВЕДЕНИЕ

В современных условиях цифровизации транспорта особое значение приобретает удобство и доступность информации о рейсах, расписаниях и услугах аэропортов, а также возможность быстрой и надежной покупки авиабилетов. Несмотря на существование различных приложений и сервисов, многие из них либо не охватывают все направления и услуги, либо не предоставляют удобного интерфейса для взаимодействия с пользователями. Это создает неудобства для пассажиров и снижает эффективность использования авиационного транспорта.

Цель данной курсовой работы — разработка информационной системы для аэропорта, которая будет представлять собой агрегатор, обеспечивающий централизованное хранение, обработку и предоставление данных о расписаниях рейсов, услугах аэропорта и возможностях покупки билетов. Такая система будет полезна пассажирам для оперативного планирования поездок, а также авиационным компаниям и аэропортам, стремящимся повысить качество обслуживания и удовлетворенность клиентов.

В рамках данной работы будут выполнены следующие задачи:

1 Анализ существующих аналогов и выявление их достоинств и недостатков.

2 Формирование функциональных требований и выбор подходящих инструментов для разработки.

3 Проектирование базы данных, включая разработку инфологической и даталогической моделей.

4 Создание физической базы данных с использованием современных технологий и инструментов.

5 Проведение тестирования для оценки работоспособности системы.

Важной частью разработки является проектирование базы данных, которая должна обеспечивать надежное хранение данных, минимизацию ошибок и высокую производительность при выполнении запросов.

Результаты данной работы могут быть использованы в авиационной отрасли для создания сервисов, улучшающих пользовательский опыт, сокращающих время на планирование поездок и покупку билетов, а также повышающих общую привлекательность авиаперелетов.

1 ОБЗОР СУЩЕСТВУЮЩИХ АНАЛОГОВ

На белорусском рынке существует множество сервисов и приложений, предоставляющих информацию о рейсах, услугах аэропортов и бронировании билетов. Эти системы отличаются функциональностью, доступностью и уровнем удобства для пользователей. Наибольшую популярность среди клиентов завоевали такие решения, как «Aviasales», «Skyscanner» и «FlightAware».

Данные сервисы обеспечивают доступ к информации о рейсах, помогают планировать поездки и отслеживать изменения в расписании. Однако каждый из них обладает как преимуществами, так и недостатками, которые необходимо учитывать при разработке нового агрегатора для аэропорта. Анализ этих аналогов позволяет выделить важные аспекты, которые следует улучшить и интегрировать в новый проект.

1.1 Aviasales

«Aviasales» – это один из крупнейших международных агрегаторов авиабилетов, предлагающий пользователям поиск и бронирование рейсов по всему миру. Сервис ориентирован на широкий круг пользователей, предоставляя удобные инструменты для планирования путешествий, включая сравнение цен и маршрутов. [1].

Функциональность:

1 Поиск авиабилетов по заданным параметрам: Пользователь вводит данные о точке вылета, пункте назначения, дате и количестве пассажиров. Система анализирует множество предложений от авиакомпаний и агрегаторов, чтобы предоставить максимально релевантные варианты. Сравнение цен на авиабилеты от различных авиакомпаний и агентов.

2 Фильтры для выбора рейсов: Функция фильтрации позволяет настроить параметры поиска, такие как количество пересадок, время вылета и прилета, длительность перелета, класс обслуживания, а также выбор конкретных авиакомпаний. Встроенная система уведомлений о снижении цен.

3 Гибкий календарь цен: Уникальная функция, показывающая стоимость билетов на разные даты, что позволяет подобрать наиболее экономичный вариант для путешествия.

4 Отслеживание цен: Пользователь может настроить уведомления об изменении цен на интересующий маршрут. Система автоматически сообщает, если стоимость билетов увеличилась или снизилась.

Преимущества:

1 Широкая база данных авиарейсов: Aviasales анализирует миллионы предложений от множества авиакомпаний, включая как крупных международных перевозчиков, так и бюджетные авиалинии. Это обеспечивает пользователям огромный выбор маршрутов и цен. Удобный интерфейс и

фильтры. Пользователи могут быстро настроить параметры поиска для нахождения оптимального рейса.

2 Экономия времени и денег: благодаря продуманным инструментам поиска и фильтрации, пользователи могут быстро найти выгодные предложения без необходимости посещать десятки сайтов авиакомпаний.

3 Доступ к скрытым скидкам: Сервис предоставляет специальные предложения и скидки, доступные только через Aviasales, что делает его особенно привлекательным для экономных путешественников.

4 Интуитивно понятный интерфейс: Удобный интерфейс позволяет даже новичкам в путешествиях легко находить и бронировать рейсы.

Недостатки:

1 Отсутствие прямого бронирования билетов: Сервис не занимается продажей билетов напрямую. Он перенаправляет пользователей на сайты авиакомпаний или партнерских агентств, где и осуществляется оплата.

2 Комиссии партнеров: Некоторые партнеры добавляют комиссии за бронирование, что увеличивает итоговую стоимость билетов. Пользователь не всегда может узнать об этом заранее.

3 Ограниченная поддержка пользователей: так как Aviasales выступает только как поисковая система, все вопросы по изменениям или отмене бронирования приходится решать непосредственно с авиакомпаниями или агентствами, что может быть неудобно.

Таким образом, «Aviasales» — это мощный инструмент для поиска и бронирования авиабилетов, предоставляющий огромный выбор маршрутов и цены. Его удобство использования и широкий функционал делают сервис востребованным среди путешественников по всему миру.

Однако при разработке нового агрегатора стоит обратить внимание на устранение ключевых недостатков Aviasales, таких как отсутствие прямого бронирования и зависимость от партнеров. Например, добавление функции прямой покупки билетов и расширение поддержки пользователей могут значительно повысить лояльность аудитории.

1.2 Skyscanner

«Skyscanner» – это один из самых популярных международных поисковиков авиабилетов, гостиниц и аренды автомобилей. Сервис предоставляет пользователям возможность сравнить цены и выбрать лучшие предложения для путешествий. Он отличается универсальностью, простотой интерфейса и надежностью информации. [2].

Функциональность:

1 Поиск авиабилетов: Skyscanner предоставляет возможность искать авиабилеты по различным параметрам: пункт отправления и назначения, даты, количество пассажиров, класс обслуживания. Система анализирует предложения от авиакомпаний, агентств и агрегаторов, предоставляя пользователю самые актуальные варианты. Сравнение цен на билеты и другие услуги.

2 Поиск гостиниц: Интеграция с сервисами бронирования отелей позволяет искать варианты размещения, фильтровать их по рейтингу, стоимости, удаленности от центра города или достопримечательностей. Интерактивная карта с доступными рейсами.

3 Аренда автомобилей: Пользователи могут сравнивать предложения по аренде автомобилей в различных городах и странах, что упрощает планирование наземного передвижения.

4 Отслеживание цен: Skyscanner позволяет настроить уведомления об изменении цен на интересующие рейсы. Это помогает сэкономить деньги, выбрав оптимальный момент для бронирования.

5 Календарь низких цен: Функция позволяет пользователям видеть цены на рейсы в пределах месяца, помогая найти самые дешевые даты для перелетов.

Преимущества:

1 Всеобъемлющий поиск: Skyscanner охватывает миллионы предложений от авиакомпаний, агентств и других сервисов. Это делает его универсальным инструментом для путешественников, которым важно иметь максимальный выбор.

2 Удобный интерфейс. Простота использования и интуитивная навигация делают сервис доступным для широкого круга пользователей.

3 Прозрачность: Skyscanner предоставляет полную информацию о стоимости, включая налоги и сборы. Пользователь может сразу увидеть реальную цену, без скрытых платежей.

4 Удобный интерфейс: Сервис предлагает минималистичный, интуитивно понятный дизайн, который делает процесс поиска и бронирования быстрым и приятным.

5 Инновационные функции: Функция "Везде" и календарь цен помогают пользователям находить выгодные предложения, даже если они еще не определились с точным маршрутом или датами.

Недостатки:

1 Отсутствие прямого бронирования: как и Aviasales, Skyscanner не занимается продажей билетов. Пользователь перенаправляется на сайты партнеров, где может столкнуться с дополнительными комиссиями или проблемами с оплатой.

2 Качество данных от партнеров: иногда информация о рейсах или ценах может быть неактуальной, что связано с задержкой обновлений у партнеров.

3 Ограниченная поддержка: Все вопросы, связанные с изменением или отменой билетов, необходимо решать через сайты партнеров. Это может быть неудобно для пользователей, предпочитающих единый сервис.

4 Сложность возвратов: поскольку Skyscanner выступает только как агрегатор, процесс возврата билетов или изменения бронирования полностью зависит от условий партнеров, что может усложнить взаимодействие.

«Skyscanner» — это надежный и удобный инструмент для поиска авиабилетов, отелей и аренды автомобилей. Его функциональность и доступность делают его особенно популярным среди путешественников, которые ценят удобство планирования поездок.

Однако при разработке нового сервиса стоит обратить внимание на улучшение таких аспектов, как прямое бронирование, актуализация данных и усиление поддержки пользователей.

Например, добавление функции кэширования данных для предотвращения ценовых расхождений или создание "личного кабинета" для управления бронированиями может стать серьезным конкурентным преимуществом.

1.3 FlightAware

«FlightAware» — это популярная платформа, предоставляющая в реальном времени данные о полетах и авиационной деятельности. Она ориентирована как на путешественников, так и на профессионалов в авиационной индустрии.

Сервис специализируется на отслеживании рейсов, аналитике и предоставлении информации об аэропортах и авиакомпаниях.

Функциональность:

1 Отслеживание рейсов: FlightAware позволяет отслеживать коммерческие, чартерные и даже частные рейсы в реальном времени. Пользователь может вводить номер рейса, маршрут или аэропорт, чтобы получить данные о местоположении самолета, его скорости, высоте, статусе (в пути, задержан, прибыл).

2 Карта в реальном времени: Визуализация маршрутов полетов на интерактивной карте с возможностью видеть не только конкретный рейс, но и общую картину воздушного движения в регионе.

3 Данные о погоде: Платформа интегрирует метеорологическую информацию, что помогает оценить влияние погодных условий на рейс. Это особенно полезно для пилотов, авиакомпаний и любителей авиации.

4 Оповещения и уведомления: Пользователи могут настроить автоматические уведомления о статусе рейсов (изменение времени вылета, посадки, отмена или значительные задержки).

5 Информация об аэропортах: FlightAware отображает статус работы аэропортов, загруженность, взлетно-посадочные полосы и статистику задержек. Это может быть полезно для людей, планирующих пересадки или работающих в авиационной логистике.

6 Отзывы пользователей о перевозчиках и рейсах.

Преимущества:

1 Высокая точность данных: FlightAware использует множество источников, включая данные авиакомпаний, аэропортов, ADS-B ресиверы, чтобы предоставлять максимально точную информацию о полетах.

2 Глобальное покрытие: Сервис отслеживает рейсы по всему миру, включая отдаленные регионы и рейсы малой авиации, что делает его универсальным инструментом.

3 Бесплатный и премиум-доступ: FlightAware предоставляет основные функции бесплатно, а профессиональные пользователи могут воспользоваться подпиской для получения углубленной аналитики и API-доступа.

4 Пользовательские уведомления: Возможность получать уведомления о рейсах в реальном времени делает сервис удобным для пассажиров и их встречающих.

Недостатки:

1 Ограничения бесплатной версии: Бесплатная версия предоставляет ограниченный доступ к аналитике и деталям рейсов. Многие функции, такие как API-доступ, доступны только в премиум-подписке.

2 Ограниченное количество уведомлений: В бесплатной версии число оповещений о рейсах ограничено, что может быть неудобным для пользователей, которым требуется отслеживать несколько рейсов.

3 Сложный интерфейс для новичков: Новым пользователям может быть сложно разобраться в интерфейсе, особенно в деталях аналитики и работе с картой.

4 Платные функции для профессионалов: Продвинутые функции, такие как детализация полетов или углубленная аналитика, доступны только за дополнительную плату, что может ограничивать использование сервиса небольшими компаниями или индивидуальными пользователями.

«FlightAware» — это уникальная платформа для отслеживания рейсов и анализа данных об авиации. Она подходит как профессионалам, так и любителям.

При разработке нового сервиса можно взять на вооружение сильные стороны FlightAware, такие как интеграция с ADS-B, оповещения и информативные карты. Однако можно улучшить доступность функционала для обычных путешественников, например, добавив более простой интерфейс и базовую аналитику в бесплатной версии.

2 ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ И ИНСТРУМЕНТЫ РАЗРАБОТКИ

2.1 Функциональные требования к системе

Основной задачей разрабатываемого агрегатора авиабилетов является создание эффективного и удобного сервиса для пользователей, позволяющего искать, бронировать и оплачивать авиабилеты, а также получать информацию о рейсах и услугах аэропортов. Для реализации этого необходимо определить ключевые функциональные требования, которые обеспечат необходимую функциональность системы.

Основные функциональные требования к приложению:

1 Поиск рейсов. Пользователи должны иметь возможность искать доступные авиарейсы по точке вылета и назначения, а также по дате и времени отправления и прибытия. Необходимо реализовать фильтрацию рейсов по различным параметрам, таким как стоимость билета, длительность перелета, авиакомпания, класс обслуживания и другие критерии.

2 Бронирование билетов. Система должна предоставлять возможность бронирования авиабилетов, включая выбор места в салоне самолета. Пользователи должны иметь доступ к интерактивной схеме посадочных мест, где можно выбрать и забронировать место онлайн.

3 Оплата билетов. Платформа должна поддерживать онлайн-оплату с использованием различных методов: банковских карт, электронных кошельков, платежных систем и других. Для международных рейсов важна поддержка мультивалютных операций, а также обеспечение безопасности транзакций.

4 Управление учетными записями пользователей. Пользователи должны иметь возможность регистрироваться в системе, редактировать свои личные данные, добавлять информацию о документах, необходимых для полетов, а также просматривать историю поездок и сохраненные маршруты. Система должна включать механизмы аутентификации и авторизации, возможность восстановления пароля и подтверждения учетной записи.

5 Информация о рейсах и аэропортах. Приложение должно предоставлять информацию о статусе рейсов (время вылета, прибытия, задержки, отмены), номерах терминалов и выходов на посадку. Также необходимо отображать данные об инфраструктуре аэропортов: расположение стоек регистрации, зоны отдыха, магазины и кафе.

6 Система лояльности. Разработка системы лояльности для пользователей, где за покупку билетов можно накапливать баллы, которые можно обменивать на скидки, апгрейд класса обслуживания или доступ к премиальным услугам. Система должна учитывать различные уровни лояльности с дополнительными бонусами для активных пользователей.

7 Система отзывов и рейтингов. Пользователи должны иметь

возможность оставлять отзывы о перелетах, качестве обслуживания авиакомпаний и инфраструктуре аэропортов. Система должна отображать рейтинги авиакомпаний и рейсов на основе отзывов пользователей.

8 Поддержка нескольких языков. Для обеспечения удобства пользователей из разных стран система должна поддерживать многоязычный интерфейс, включая русский, английский и другие языки в зависимости от региональной аудитории.

9 Интеграция с внешними сервисами. Возможность интеграции с другими платформами для получения актуальной информации о рейсах, ценах на билеты, а также работы с дополнительными функциями, такими как бронирование отелей, аренда автомобилей и страховые услуги. Необходима интеграция с платежными системами и метеорологическими сервисами для отображения данных о погоде в пункте назначения.

Таким образом, для эффективной работы агрегатора аэропортов необходимо обеспечить комплексную функциональность, включающую поиск и фильтрацию рейсов, возможность бронирования и оплаты билетов, управление учетными записями пользователей и внедрение системы лояльности. Важным элементом является система отзывов и рейтингов, поддержка нескольких языков и интеграция с внешними сервисами. Все эти требования направлены на создание удобного и высокоэффективного сервиса, предоставляющего актуальную информацию, удобство бронирования и оплаты билетов, а также поддержку пользователей на всех этапах взаимодействия с системой.

2.2 Выбор СУБД

Для реализации проекта аэропорт был выбран подход, основанный на использовании реляционной базы данных (СУБД), так как предполагается работа с большими объемами структурированных данных, которые логично представлять в виде таблиц, связанных между собой. Реляционная модель данных идеально подходит для хранения информации о пользователях, маршрутах, транзакциях и бронированиях, поскольку она обеспечивает четкую организацию данных и позволяет эффективно их обрабатывать [4].

Требования к СУБД:

1 Поддержка работы с большими объемами данных. В системе будет храниться большое количество данных о пользователях, рейсах, маршрутах и транзакциях. Важно, чтобы СУБД обеспечивала высокую производительность при работе с большими объемами информации, а также быстрое выполнение операций вставки, обновления и удаления данных.

2 Поддержка транзакций. Транзакции играют важную роль в системе, особенно в части бронирования и оплаты билетов. Важно, чтобы СУБД гарантировала целостность данных в случае ошибок или сбоев, а также поддерживала механизмы отката транзакций. Система должна удовлетворять ACID-свойствам (атомарность, согласованность, изолированность,

долговечность) для обеспечения надежности работы с финансовыми операциями.

3 Высокая производительность при выполнении сложных запросов. Важной частью системы является возможность быстрого поиска и фильтрации маршрутов по различным параметрам: времени отправления, стоимости, перевозчику и т.д.

4 Поддержка масштабируемости и расширяемости. Система должна быть готова к масштабированию, так как в будущем может потребоваться поддержка большего количества пользователей, маршрутов и дополнительных функциональностей. Важно, чтобы СУБД позволяла эффективно увеличивать объемы данных без потери производительности.

5 Возможности для интеграции с внешними сервисами. В системе будет интеграция с внешними сервисами, например, для осуществления платежей или получения актуальной информации о рейсах. Поэтому СУБД должна поддерживать возможность подключения и взаимодействия с внешними API и системами.

Для реализации проекта аэропорт были рассмотрены несколько популярных реляционных СУБД. Каждый из этих вариантов обладает своими преимуществами и недостатками, которые следует учитывать при выборе оптимального решения для системы. В процессе выбора СУБД учитывались такие факторы, как поддержка работы с большими объемами данных, транзакциями, производительность и масштабируемость. Ниже подробно рассматриваются четыре СУБД, которые были анализированы для данного проекта.

2.2.1 PostgreSQL

PostgreSQL — это одна из самых мощных и гибких реляционных СУБД с открытым исходным кодом. Ее история начинается в 1986 году, когда профессор Майкл Стоунбрейкер из Университета Калифорнии в Беркли начал разработку системы под названием POSTGRES, которая была предназначена для исследования теоретических концепций в области реляционных баз данных. В 1996 году POSTGRES был переименован в PostgreSQL, и с тех пор эта СУБД развивается как мощный инструмент с открытым исходным кодом, который активно используется для решения задач, связанных с хранением и обработкой больших объемов данных [5].

Преимущества:

1 Высокая производительность при работе с большими объемами данных и сложными запросами. PostgreSQL предоставляет отличную производительность при работе с большими объемами данных и сложными запросами. Это достигается за счет использования индексации, оптимизации запросов и эффективной работы с большими таблицами. В контексте проекта аэропорт, где необходимо обрабатывать большое количество маршрутов, транзакций и запросов, PostgreSQL обеспечит быструю и стабильную работу даже при высоких нагрузках.

2 PostgreSQL полностью поддерживает транзакции и гарантирует соблюдение ACID-свойств. Это особенно важно для системы бронирования билетов, где каждая транзакция должна быть выполнена точно и корректно, чтобы избежать ошибок в данных, например, при одновременной попытке забронировать одно и то же место двумя пользователями. С помощью транзакций обеспечивается целостность данных, что делает систему надежной и безопасной.

3 Возможность работы с сложными типами данных. PostgreSQL поддерживает работу с широким спектром сложных типов данных, включая JSON, массивы, хранимые процедуры и пользовательские типы данных. Это позволяет эффективно хранить и манипулировать неструктурированными данными, такими как информация о маршрутах, местах в самолете и прочее. В контексте аэропорта, это может быть полезно для динамичного хранения данных, таких как расписания, информация о бронированиях и другие параметры, которые могут варьироваться.

4 Поддержка расширений для географических данных и полнотекстового поиска. PostgreSQL имеет встроенную поддержку расширений, таких как PostGIS, для работы с географическими данными. Это расширение идеально подходит для систем, которые работают с картами и маршрутами, как в случае с аэропортом. Возможность выполнения полнотекстового поиска также позволяет значительно улучшить функциональность поиска по маршрутам и маршрутам перевозчиков, что делает систему более удобной для пользователей.

5 Бесплатная и открытая лицензия. PostgreSQL распространяется по

лицензии Open Source, что означает отсутствие платы за лицензию и возможность свободного использования, модификации и распространения кода. Это снижает затраты на лицензирование, что особенно важно для стартапов и компаний с ограниченными бюджетами. Бесплатность PostgreSQL делает его доступным для широкого круга пользователей и предоставляет большую гибкость в разработке системы.

Недостатки:

1 Требуется высокой квалификации для настройки и оптимизации. PostgreSQL — это мощная СУБД, которая требует определенных знаний и навыков для оптимальной настройки и эксплуатации. Например, необходимо уметь настраивать индексы, параллельные запросы и систему репликации для достижения наилучших результатов. Без достаточного опыта в настройке и управлении СУБД возможны проблемы с производительностью и надежностью системы.

2 Сложность развертывания и настройки по сравнению с другими СУБД. В отличие от более простых СУБД, таких как MySQL или SQLite, развертывание PostgreSQL может быть более сложным. Это связано с множеством настроек, параметров и возможностей, которые нужно учитывать при первоначальной настройке базы данных. Настройка масштабируемости, репликации и резервного копирования также может потребовать значительных усилий и времени для достижения нужного уровня эффективности и безопасности.

Таким образом, несмотря на более сложное развертывание и необходимость в высококвалифицированных специалистах для настройки, PostgreSQL представляет собой отличное решение для проектов, которые требуют обработки больших объемов данных и сложных операций, таких как аэропорт.

2.2.2 MySQL

MySQL — это одна из самых популярных и широко используемых реляционных систем управления базами данных с открытым исходным кодом. История MySQL началась в 1994 году, когда шведская компания MySQL AB, основанная Михаэлем Видениусом и Альфом Ларссоном, начала разрабатывать эту СУБД как решение для работы с большими объемами данных, ориентированное на интернет-приложения. В 2008 году компанию MySQL AB приобрела корпорация Sun Microsystems, а спустя два года — Oracle Corporation. Несмотря на смену владельцев, MySQL продолжает оставаться одной из самых популярных СУБД в мире [6].

Преимущества:

1 Простота в использовании и настройке. MySQL известен своей простотой в установке и настройке. Это особенно важно для разработчиков, которым необходимо быстро развернуть систему и начать работать с базой данных. В отличие от более сложных СУБД, таких как PostgreSQL, MySQL не требует глубокой настройки и специфичных знаний для начальной конфигурации. Это делает его хорошим выбором для стартапов и небольших проектов, где важна скорость развертывания.

2 Хорошая поддержка репликации и масштабируемости. MySQL предлагает надежную систему репликации, которая позволяет создавать резервные копии данных и эффективно распределять нагрузку между несколькими серверами. Это критически важно для обеспечения высокой доступности и масштабируемости системы. Репликация помогает в создании системы с высокой отказоустойчивостью, что также улучшает производительность при высоких нагрузках. В контексте аэропорта, MySQL может эффективно справляться с ростом числа пользователей и объемом данных, обеспечивая бесперебойную работу при увеличении нагрузки.

3 Широкая поддержка сообщества и разработчиков. MySQL имеет большое и активное сообщество разработчиков, что облегчает решение проблем и ускоряет процесс поиска решений для возникающих вопросов. Большое количество готовых библиотек, инструментов и документации позволяет быстро адаптироваться к работе с базой данных. Это снижает затраты на поддержку и делает систему более доступной для разработчиков с разным уровнем подготовки.

Недостатки:

1 Меньшая производительность для работы с большими объемами данных по сравнению с PostgreSQL. Несмотря на хорошую производительность MySQL, он уступает PostgreSQL в работе с большими объемами данных и сложными запросами. PostgreSQL лучше подходит для обработки больших наборов данных, сложных выборок и транзакций, что делает его более подходящим для систем, работающих с крупными базами данных и требующих высокой производительности. В случае с аэропортом, где необходимо эффективно обрабатывать множество рейсов и бронирований,

MySQL может проявить себя хуже при выполнении сложных запросов, таких как поиск по нескольким критериям или расчет стоимости.

2 Ограниченная поддержка сложных типов данных и некоторых расширений. MySQL имеет ограниченную поддержку для работы с более сложными типами данных, такими как JSON или массивы, по сравнению с PostgreSQL. Также в MySQL отсутствует поддержка ряда расширений, таких как PostGIS для географических данных или полнотекстового поиска на уровне PostgreSQL. Для проекта аэропорт это может стать ограничением, если потребуется хранить и обрабатывать данные, которые не подходят для традиционных реляционных типов данных (например, географическая информация о маршрутах или данные в формате JSON).

Таким образом, MySQL представляет собой отличное решение для простых и средних по сложности проектов, где важна высокая доступность, масштабируемость и быстрая настройка. Однако для более сложных проектов, таких как аэропорт, где требуется высокая производительность при работе с большими объемами данных и поддержка сложных типов данных, PostgreSQL может быть более подходящим выбором.

2.2.3 Oracle Database

Oracle Database — это одна из самых мощных и широко используемых коммерческих реляционных систем управления базами данных, разработанная компанией Oracle Corporation. История Oracle началась в 1977 году, когда Ларри Эллисон, Боб Минтер и Эд Оутс основали компанию Relational Software Inc., которая позже была переименована в Oracle Corporation. В 1979 году Oracle представила первую коммерческую реляционную СУБД, которая использовала язык запросов SQL (Structured Query Language). С тех пор Oracle стал лидером на рынке корпоративных СУБД, с особым акцентом на высокую производительность, надежность и масштабируемость, что сделало Oracle Database стандартом для крупных организаций и финансовых учреждений [7].

Преимущества:

1 Высокая производительность и отличная масштабируемость. Oracle Database предоставляет высокую производительность, особенно для крупных корпоративных приложений, работающих с большими объемами данных. Система способна обрабатывать огромные нагрузки и работать с большими базами данных, что делает ее идеальной для систем, которые требуют высокой скорости обработки данных при выполнении сложных транзакций и запросов. Это преимущество особенно важно для крупных агрегаторов с большим количеством пользователей и сложной структурой данных.

2 Oracle Database известна своей надежной поддержкой транзакций и соблюдением ACID-свойств, что гарантирует целостность данных. Это особенно важно в контексте системы бронирования, где важно обеспечить, чтобы данные о бронированиях и оплатах были точными и не были потеряны при сбоях системы. Также поддержка репликации данных позволяет создавать

высокодоступные решения, обеспечивая бесперебойную работу системы.

3 Широкий спектр функциональных возможностей для бизнеса. Oracle Database предоставляет множество инструментов для бизнес-анализа, работы с большими данными, машинного обучения и аналитики. Это делает систему универсальным инструментом для предприятий, которые могут использовать эти функции для углубленного анализа данных и построения бизнес-отчетности. Для проекта аэропорт это может быть полезно, если система будет расширяться с внедрением аналитики для прогнозирования спроса на билеты или изучения поведения пользователей.

Недостатки:

1 Высокая стоимость лицензирования. Одним из главных недостатков Oracle Database является высокая стоимость лицензирования, что делает ее дорогим решением для малых и средних бизнесов. Стоимость лицензий и ежегодной поддержки может быть значительным бременем для стартапов и небольших компаний, особенно если они не используют все возможности СУБД. В контексте проекта аэропорт это может означать большие затраты на эксплуатацию системы, которые могут быть не оправданы для небольшого бизнеса.

2 Требуется значительных ресурсов для развертывания и эксплуатации. Развертывание и эксплуатация Oracle Database требуют значительных вычислительных и человеческих ресурсов. Это включает в себя не только покупку и установку серверов, но и поддержку на протяжении всего срока эксплуатации. Сложная настройка и оптимизация базы данных могут потребовать высококвалифицированных специалистов, что увеличивает затраты на техническую поддержку. Для небольших проектов это может быть чрезмерным, особенно если задачи можно решить с помощью более легких и дешевых СУБД.

3 Не является бесплатной. В отличие от PostgreSQL или MySQL, Oracle Database является платной СУБД с ограниченной бесплатной версией. Стоимость лицензирования может быть высока, что увеличивает затраты на проект и делает решение менее привлекательным для стартапов и малых компаний. Для аэропорта, где высокая стоимость лицензии может не быть оправдана функциональностью, доступной в других СУБД, использование Oracle Database может быть нецелесообразным.

Таким образом, несмотря на многочисленные преимущества Oracle Database, такие как высокая производительность, поддержка транзакций и масштабируемость, для проекта аэропорт, ориентированного на малый и средний бизнес, эта СУБД может быть слишком дорогой и ресурсоемкой. В данном случае выбор более легковесной и менее затратной СУБД, такой как PostgreSQL или MySQL, может быть более оправдан.

2.2.4 SQLite

SQLite — это легковесная реляционная система управления базами

данных, которая была впервые выпущена в 2000 году. Ее создал Ричард Хипп, который разработал SQLite как встраиваемую СУБД, ориентированную на простоту использования и компактность. SQLite была спроектирована так, чтобы быть максимально легкой, быстро внедряемой и без необходимости в отдельном сервере баз данных [8].

Преимущества:

1 Простота интеграции и использования. SQLite — это серверная СУБД, которая не требует настройки отдельного серверного компонента. Это делает ее особенно удобной для интеграции в приложения. Для небольших проектов или мобильных приложений SQLite является отличным выбором, так как она не требует особых усилий для развертывания, что значительно ускоряет процесс разработки. Важно отметить, что база данных SQLite реализуется в виде одного файла, что упрощает ее использование и переносимость.

Недостатки:

1 Ограниченная масштабируемость и производительность для работы с большими объемами данных. SQLite не предназначена для работы с высокими нагрузками и большими объемами данных. Она не обладает возможностями масштабирования, которые присутствуют в более сложных СУБД, таких как PostgreSQL или MySQL. Это ограничение проявляется в снижении производительности при увеличении количества пользователей или данных. Для систем с интенсивным использованием, таких как аэропорт, где необходимо обрабатывать множество транзакций и запросов, SQLite может не быть достаточно эффективной, особенно когда речь идет о масштабировании системы.

2 Отсутствие полноценной поддержки транзакций на уровне предприятия. Несмотря на то, что SQLite поддерживает транзакции, она не предназначена для использования в крупных предприятиях с высокими требованиями к целостности и надежности данных. SQLite ограничена в функционале управления параллельными транзакциями и в обработке больших потоков данных. Это может быть критическим недостатком для систем, таких как бронирование билетов и обработка платежей, где требуется высокая степень целостности данных и возможности для обеспечения надежности при многопользовательской работе.

На основе анализа различных СУБД было принято решение использовать PostgreSQL для реализации базы данных аэропорта. Это решение обусловлено высокой производительностью PostgreSQL, поддержкой транзакций, гибкостью работы с данными и расширениями, а также бесплатной лицензией.

Эти характеристики обеспечат эффективную работу системы с большими объемами данных и сложными запросами, а также гарантируют целостность и надежность при обработке операций с билетами и оплатой.

3 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

Проектирование базы данных является важным этапом в разработке системы аэропорт, так как правильная структура данных определяет эффективность работы системы, обеспечивая быструю обработку запросов, целостность и безопасность данных. В рамках этого раздела будут рассмотрены этапы проектирования, включая инфологическую и даталогическую модели, модель миграции данных, а также требования к входным данным и форматы их хранения.

3.1 Инфологическая модель

Инфологическая модель — это этап проектирования базы данных, на котором происходит детальное описание структуры данных, их взаимосвязей и организационных единиц системы. В рамках этого этапа создается концептуальная модель, отражающая основные объекты и связи между ними. Инфологическая модель не зависит от конкретной реализации в СУБД, она служит основой для дальнейшего проектирования и реализации базы данных, позволяя понять, как данные будут структурированы и взаимодействовать в системе.

В данной работе инфологическая модель базы данных отражает структуру и связи между сущностями, необходимыми для разработки аэропорта. Основными сущностями являются пользователи, авиарейсы, билеты, транзакции лояльности и различные связанные с ними данные. Важно, чтобы модель была гибкой и могла эффективно обрабатывать запросы, касающиеся маршрутов, бронирований, лояльности и поддержки пользователей [9].

Инфологическая модель базы данных включает в себя следующие сущности и их атрибуты:

1 Пользователь (User). Атрибуты: имя пользователя, хеш пароля, email, активен ли пользователь, является ли администратором, является ли сотрудником, связь с паспортом, связь с уведомлениями, связь с отзывами, связь с событиями.

2 Национальность (Nationality). Атрибуты: название национальности. Связи: используется в паспортах, используется в экипаже.

3 Пол (Gender). Атрибуты: название пола. Связи: используется в паспортах.

4 Страна (Country). Атрибуты: название страны, код страны. Связи: используется в паспортах, используется в городах, используется в авиакомпаниях.

5 Паспорт (Passport). Атрибуты: имя, фамилия, дата рождения, номер паспорта, идентификационный номер, дата выдачи, дата истечения. Связи: национальность, пол, страна, связь с пользователем, связь с билетами.

6 Город (City). Атрибуты: название города. Связи: связь со страной, используется в аэропортах.

7 Аэропорт (Airport). Атрибуты: название аэропорта, код IATA. Связи: связь с городом, используется в маршрутах, используется в рейсах.

8 Самолет (Plane). Атрибуты: модель, вместимость, регистрационный номер, находится ли на техническом обслуживании. Связи: используется в рейсах, связь с классами мест, связь с местами.

9 Авиакомпания (Airline). Атрибуты: название авиакомпании, описание. Связи: связь со страной, используется в рейсах.

10 Рейс (Flight). Атрибуты: номер рейса. Связи: связь с аэропортом, связь с самолетом, связь с авиакомпанией, используется в расписаниях, используется в экипажах, используется в отзывах, используется в маршрутах.

11 Класс мест (SeatClass). Атрибуты: название класса. Связи: используется в местах.

12 Место (Seat). Атрибуты: номер места, ряд. Связи: связь с классами мест, связь с самолетом, используется в билетах.

13 Билет (Ticket). Атрибуты: дата бронирования, цена. Связи: связь с рейсом, связь с местом, связь с паспортом, используется в багажах, используется в питании, используется в оплатах.

14 Расписание (Schedule). Атрибуты: ожидаемое время вылета, ожидаемое время прибытия, фактическое время вылета, фактическое время прибытия. Связи: связь с рейсом.

15 Багаж (Baggage). Атрибуты: вес. Связи: связь с билетом.

16 Тип события (EventType). Атрибуты: название типа события. Связи: используется в событиях.

17 Событие (Event). Атрибуты: название события, дата и время, описание. Связи: связь с пользователем, связь с типом события.

18 Экипаж (Crew). Атрибуты: имя, фамилия, должность. Связи: связь с национальностью, используется в назначениях экипажа.

19 Назначение экипажа (CrewAssignment). Атрибуты: роль. Связи: связь с экипажем, связь с рейсом.

20 Маршрут (Route). Атрибуты: аэропорт вылета, аэропорт прибытия, расстояние. Связи: используется в рейсах.

21 Питание (Meal). Атрибуты: название блюда, описание, цена. Связи: используется в питании билетов.

22 Питание билетов (TicketMeal). Атрибуты: количество. Связи: связь с билетом, связь с блюдом.

23 Оплата (Payment). Атрибуты: дата оплаты, сумма, метод оплаты. Связи: связь с билетом.

24 Отзыв (Review). Атрибуты: рейтинг, комментарии. Связи: связь с пользователем, связь с рейсом.

25 Уведомление (Notification). Атрибуты: текст уведомления, дата создания, прочитано ли. Связи: связь с пользователем.

Таким образом, инфологическая модель для системы аэропорт

предоставляет понятное описание сущностей и их связей. Эта модель не включает технических аспектов хранения данных, но служит основой для последующего проектирования и реализации базы данных в выбранной СУБД.

3.2 Даталогическая модель

Даталогическая модель описывает структуру данных на уровне базы данных, в том числе типы данных, ограничения и связи между таблицами. Она отвечает за определение точных типов данных для каждого атрибута сущности, установление ограничений на значения и обеспечение целостности данных. В данном разделе представлена детальная спецификация всех сущностей, их атрибутов и типов данных, которые будут использоваться в базе данных системы. Модель строится на основе инфологической модели, уточняя типы данных и устанавливая точные форматы хранения информации для эффективного функционирования системы [10].

Описание таблиц и их атрибутов:

1 Пользователь (Users):

- id (SERIAL) — уникальный идентификатор пользователя;
- username (VARCHAR(45)) — имя пользователя;
- email (VARCHAR(254)) — email пользователя;
- password (VARCHAR(64)) — хеш пароля;
- is_active (BOOLEAN) — статус активности пользователя;
- is_admin (BOOLEAN) — является ли администратором;
- is_staff (BOOLEAN) — является ли сотрудником;

2 Национальность (Nationality):

- id (SERIAL) — уникальный идентификатор национальности;
- name (VARCHAR(45)) — название национальности;

3 Пол (Gender):

- id (SERIAL) — уникальный идентификатор пола;
- name (VARCHAR(45)) — название пола;

4 Страна (Country):

- id (SERIAL) — уникальный идентификатор страны;
- name (VARCHAR(45)) — название страны;
- code (VARCHAR(3)) — код страны;

5 Паспорт (Passport):

- Id (SERIAL) — уникальный идентификатор паспорта;
- first_name (VARCHAR(45)) — имя;
- last_name (VARCHAR(45)) — фамилия;
- birth_date (DATE) — дата рождения;
- pass_number (VARCHAR(45)) — номер паспорта;
- identification_number (VARCHAR(45)) — идентификационный номер;
- issue_date (DATE) — дата выдачи;
- expiry_date (DATE) — дата истечения;

6 Город (City):

- id (SERIAL) — уникальный идентификатор города;
- name (VARCHAR(45)) — название города;
- 7 Аэропорт (Airport):
 - Id (SERIAL) — уникальный идентификатор аэропорта;
 - name (VARCHAR(100)) — название аэропорта;
 - code_IATA (VARCHAR(3)) — код IATA;
- 8 Самолет (Plane):
 - Id (SERIAL) — уникальный идентификатор самолета;
 - model (VARCHAR(45)) — модель самолета;
 - capacity (INT) — вместимость;
 - registration_number (VARCHAR(20)) — регистрационный номер;
 - is_under_maintenance (BOOLEAN) — находится ли на техническом обслуживании;
- 9 Авиакомпания (Airline):
 - Id (SERIAL) — уникальный идентификатор авиакомпании;
 - name (VARCHAR(45)) — название авиакомпании;
 - description (TEXT) — описание;
- 10 Рейс (Flight):
 - Id (SERIAL) — уникальный идентификатор рейса;
 - flight_number (VARCHAR(10)) — номер рейса;
- 11 Класс мест (SeatClass):
 - id (SERIAL) — уникальный идентификатор класса мест;
 - name (VARCHAR(45)) — название класса;
- 12 Место (Seat):
 - Id (SERIAL) — уникальный идентификатор места;
 - number (VARCHAR(1)) — номер места;
 - row_num (INT) — номер ряда;
- 13 Билет (Ticket):
 - id (SERIAL) — уникальный идентификатор билета;
 - reservation_date (TIMESTAMP) — дата бронирования;
 - price (DECIMAL(10,3)) — цена;
- 14 Расписание (Schedule):
 - Id (SERIAL) — уникальный идентификатор расписания;
 - expected_departure (TIMESTAMP) — ожидаемое время вылета;
 - expected_arrival (TIMESTAMP) — ожидаемое время прибытия;
 - real_departure (TIMESTAMP) — фактическое время вылета;
 - real_arrival (TIMESTAMP) — фактическое время прибытия;
- 15 Багаж (Baggage):
 - Id (SERIAL) — уникальный идентификатор багажа;
 - weight (DECIMAL(5,3)) — вес;
- 16 Тип события (EventType):
 - Id (SERIAL) — уникальный идентификатор типа события;
 - name (VARCHAR(45)) — название типа события;
- 17 Событие (Event):

- Id (SERIAL) — уникальный идентификатор события;
 - name (VARCHAR(45)) — название события;
 - datetime (TIMESTAMP) — дата и время;
 - description (TEXT) — описание;
- 18 Экипаж (Crew):
- id (SERIAL) — уникальный идентификатор экипажа;
 - first_name (VARCHAR(45)) — имя;
 - last_name (VARCHAR(45)) — фамилия;
 - position (VARCHAR(45)) — должность;
- 19 Назначение экипажа (CrewAssignment):
- id (SERIAL) — уникальный идентификатор назначения экипажа;
 - role (VARCHAR(45)) — роль;
- 20 Маршрут (Route):
- id (SERIAL) — уникальный идентификатор маршрута;
 - distance_km (DECIMAL(8,2)) — расстояние в километрах;
- 21 Питание (Meal):
- Id (SERIAL) — уникальный идентификатор питания;
 - name (VARCHAR(45)) — название блюда;
 - description (TEXT) — описание;
 - price (DECIMAL(5,2)) — цена;
- 22 Питание билетов (TicketMeal):
- id (SERIAL) — уникальный идентификатор питания билета;
 - quantity (INT) — количество;
- 23 Оплата (Payment):
- id (SERIAL) — уникальный идентификатор оплаты;
 - payment_date (TIMESTAMP) — дата оплаты;
 - amount (DECIMAL(10,2)) — сумма;
 - payment_method (VARCHAR(45)) — метод оплаты;
- 24 Отзыв (Review):
- id (SERIAL) — уникальный идентификатор отзыва;
 - rating (INT) — рейтинг;
 - comments (TEXT) — комментарии;
- 25 Уведомления (Notifications):
- id (SERIAL) — уникальный идентификатор уведомления;
 - message (TEXT) — текст уведомления;
 - created_at (TIMESTAMP) — дата создания;
 - is_read (BOOLEAN) — статус прочтения;

Разработка даталогической модели является важным этапом проектирования базы данных, так как она обеспечивает основу для дальнейшей реализации системы. В данном разделе была представлена детальная структура данных, где для каждой сущности были определены атрибуты с точными типами данных и ограничениями, а также установлены связи между таблицами. Это позволяет обеспечить целостность и согласованность данных, предотвращая ошибки при взаимодействии с базой

данных. Такая модель является основой для создания физической базы данных и дальнейшего развития системы. Правильное определение типов данных и отношений между сущностями значительно повышает производительность работы с данными и делает систему более надежной.

3.3 Модель миграции данных

В процессе реализации системы возможны ситуации, когда потребуется перенос данных из существующих источников (например, старой базы данных) в новую систему. Это может происходить по причине перехода на новую версию системы, интеграции с другими системами или улучшения структуры данных. Модель миграции данных для данного проекта будет включать в себя подготовку данных, трансформацию их в нужный формат и перенос в целевую систему, что гарантирует соответствие всех данных новым стандартам и структурам.

Миграция данных в данном контексте будет включать несколько ключевых этапов:

1 Анализ текущих данных. На первом этапе необходимо понять структуру текущих данных и определить, какие данные будут мигрированы. Для этого нужно провести анализ всех таблиц, которые будут переноситься, и их связи.

2 Извлечение данных. На этом этапе данные извлекаются из старой базы данных. Для каждой таблицы будут созданы отдельные запросы на извлечение данных. Извлечение может происходить с учетом определенных фильтров или периодичности. Также могут быть использованы инструменты для миграции данных, такие как Talend или Apache Nifi, для автоматизации процесса.

3 Загрузка данных в целевую систему. После того как данные будут извлечены и преобразованы, их необходимо загрузить в новую систему. Это будет включать выполнение SQL-запросов на вставку данных в целевые таблицы. Если данные поступают из разных источников, может быть использована автоматизация для загрузки их в соответствующие таблицы.

4 Пост-миграционный анализ и валидация. После загрузки данных важно провести валидацию на целостность и корректность: проверка наличия всех записей, сравнение количества записей в исходной и целевой таблице, проверка связей между таблицами.

Миграция данных — это неотъемлемая часть любого проекта, связанного с изменением структуры или обновлением базы данных. Для успешной миграции данных необходимо внимательно продумать все этапы — от анализа текущих данных до загрузки и проверки целостности в новой системе. Правильная реализация модели миграции данных обеспечит бесперебойную работу системы, сохранность и доступность данных, что критически важно для пользователей и бизнеса.

4 РАЗРАБОТКА БАЗЫ ДАННЫХ

Разработка базы данных – это центральный элемент построения системы, обеспечивающий основу для хранения, обработки и анализа данных. На данном этапе проектирования осуществляется переход от логической модели данных, представляющей концептуальное описание системы, к физической модели, реализующей это описание на уровне реляционной базы данных.

Процесс разработки базы данных включает несколько этапов: проектирование структуры таблиц, реализация связей между ними, настройка ограничений целостности данных, создание индексов для ускорения запросов, разработка триггеров и хранимых процедур для автоматизации операций, а также внедрение механизмов защиты данных. В рамках системы аэропорт база данных играет ключевую роль, так как содержит всю необходимую информацию о пользователях, авиарейсах, билетах, точках отправления и прибытия, самолетах и пилотах.

Разработка велась с использованием системы управления базами данных (СУБД) PostgreSQL, которая была выбрана за её высокую производительность, широкие возможности работы с реляционными данными, а также встроенные механизмы масштабирования и обеспечения безопасности.

4.1 Физическая модель базы данных

Физическая модель представляет собой описание структуры данных, которое реализуется в виде таблиц и связей между ними. Она учитывает требования к производительности системы, предотвращению избыточности данных, а также обеспечению согласованности и целостности.

В системе аэропорт предусмотрены следующие основные таблицы:

1 Таблица пользователей (users) – хранит информацию о пользователях системы, включая логин, пароль, email и роли (активен, администратор и персонал).

2 Таблица паспортов (passport) – содержит паспортные данные пользователей, включая имя, фамилию, дату рождения, номер паспорта и другие данные. Эта таблица связана с пользователями через внешний ключ (рисунок 4.1).

3 Таблица полетов (route) – хранит информацию о рейсах, включая номер рейса, аэропорт, самолет и авиакомпанию. Это важная таблица для отслеживания информации о полетах.

4 Таблица билетов (ticket) – связана с рейсами и пассажирами, содержит данные о бронированиях, цене и привязке к паспорту и сиденью. Используется для управления билетами и бронированиями.

5 Таблица посадочных мест (seat) – хранит информацию о местах в

самолетах, включая номер сиденья и класс. Эта таблица необходима для управления распределением мест на рейсах.

6 Таблица платежей (payment) – содержит информацию о платежах за билеты, включая дату и сумму. Это таблица для управления финансовыми операциями, связанными с покупкой билетов.

```
42  -- Table passport
43  CREATE TABLE passport (
44      id SERIAL PRIMARY KEY,
45      first_name VARCHAR(45) NOT NULL,
46      last_name VARCHAR(45) NOT NULL,
47      birth_date DATE NOT NULL,
48      pass_number VARCHAR(45) NOT NULL,
49      identification_number VARCHAR(45) NOT NULL,
50      issue_date DATE NOT NULL,
51      expiry_date DATE NOT NULL,
52      nationality_id INT NOT NULL REFERENCES nationality(id),
53      gender_id INT NOT NULL REFERENCES gender(id),
54      country_id INT NOT NULL REFERENCES country(id),
55      user_id INT REFERENCES users(id) ON DELETE SET NULL,
56      CHECK (expiry_date > issue_date)
57  );
58
```

Рисунок 4.1 – Пример физической реализации таблицы паспортов

4.2 Создание таблиц

Создание таблиц является ключевым шагом в процессе реализации физической модели базы данных. На данном этапе описывается структура каждой таблицы: её поля, их типы данных, ограничения, а также связи с другими таблицами.

Каждое поле таблицы имеет определенный тип данных. Например, числовые значения, такие как стоимость или расстояние, реализуются через тип NUMERIC, строки – через VARCHAR или TEXT, а даты и время – через TIMESTAMP. Для повышения производительности и целостности данных были также настроены ограничения на уровне таблиц:

1 Поля с обязательными значениями помечены как NOT NULL, что исключает возможность хранения записей с пустыми критически важными данными.

2 Первичные ключи (PRIMARY KEY) используются для идентификации каждой строки, обеспечивая уникальность каждой записи в пределах таблицы.

3 Внешние ключи (FOREIGN KEY) обеспечивают связь между таблицами, что позволяет поддерживать логические отношения между

сущностями базы данных.

4 Уникальные ограничения (UNIQUE) предотвращают дублирование данных, гарантируя, что определенные столбцы будут содержать только уникальные значения.

На рисунке 4.2 приведен пример создания таблицы для хранения данных о пользователях. Она содержит уникальный идентификатор пользователя, информацию о ФИО пользователя, различные флаги состояния пользователя.

```
5
6  -- Table "user"|
7  CREATE TABLE users (
8      id SERIAL PRIMARY KEY,
9      username VARCHAR(45) NOT NULL UNIQUE,
10     password VARCHAR(64) NOT NULL,
11     email VARCHAR(254) NOT NULL UNIQUE,
12     is_active BOOLEAN NOT NULL DEFAULT TRUE,
13     is_admin BOOLEAN NOT NULL DEFAULT FALSE,
14     is_staff BOOLEAN NOT NULL DEFAULT FALSE,
15     CHECK (LENGTH(username) > 0),
16     CHECK (LENGTH(password) > 0)
17 );
18
```

Рисунок 4.2 – Таблица пользователей

Создание таблиц является важным этапом разработки базы данных, так как правильная структура данных обеспечивает эффективное хранение и обработку информации в процессе работы системы. В листинге А.1 представлен код для создания всех таблиц базы данных, используемых в проекте.

4.3 Оптимизация производительности

Для обеспечения высокой производительности системы и быстрого отклика на запросы пользователей в базе данных были внедрены индексы. Индексы играют важную роль в оптимизации работы с большими объемами данных, позволяя существенно сократить время, необходимое для выполнения операций выборки, поиска, сортировки и фильтрации

Без использования индексов, при выполнении запросов к базе данных, система должна просматривать все строки таблицы, что значительно увеличивает время выполнения операций. Индексы позволяют ускорить этот процесс, создавая структуры данных, которые позволяют базе данных находить нужные записи значительно быстрее. В зависимости от типа индекса и структуры данных, поиск может быть выполнен за время, пропорциональное логарифму от количества строк, что значительно быстрее линейного поиска.

Кроме того, индексы играют ключевую роль при соединении таблиц. Когда нужно выполнить объединение данных из разных таблиц, индексы помогают быстро находить соответствующие записи, что значительно повышает скорость выполнения сложных запросов с множеством соединений.

В проекте использованы следующие типы индексов:

1 Индексы типа B-tree (B-дерево) — применяются для операций сортировки и фильтрации по числовым и строковым данным. Эти индексы эффективны для диапазонных запросов и часто используются в таких полях, как дата регистрации, время отправления и другие, где требуется сортировка или фильтрация по значениям.

2 Hash-индексы — используются для точного поиска по уникальным значениям. Они оптимальны для быстрого поиска по полям с уникальными значениями, таким как имя пользователя или электронная почта, где важен точный поиск без диапазонных операций.

3 Индексы GIN (Generalized Inverted Index) — предназначены для работы с текстовыми данными и позволяют эффективно искать по подстрокам или фрагментам текста. Эти индексы применяются для полей с большими текстовыми данными, такими как описание запросов, где требуется быстрый поиск по ключевым словам.

Таким образом, внедрение индексов в базу данных не только ускоряет выполнение базовых операций, но и способствует улучшению общей производительности системы, обеспечивая эффективное использование ресурсов при работе с большими объемами данных. Скрипт для создания индексов представлен в листинге А.2.

4.4 Триггеры и процедуры

В данном разделе описаны разработанные триггеры и процедуры, обеспечивающие автоматизацию операций в базе данных, контроль целостности данных, а также повышение уровня удобства и производительности системы. Код создания триггеров и процедур представлен в листинге А.3. В проекте реализованы следующие ключевые механизмы:

1 Получение количества активных пользователей. Для автоматического получения количества активных пользователей в системе была создана функция `get_active_users_count`. Эта функция возвращает количество пользователей, у которых установлен флаг `is_active` в значение `TRUE`. Она используется для анализа активных пользователей в базе данных и позволяет поддерживать актуальные данные о пользователях системы.

2 Проверка наличия пассажира с данным паспортом. Для проверки существования пассажира с определенным паспортом была разработана функция `check_passport_exists`. Эта функция принимает номер паспорта в качестве параметра и возвращает `TRUE`, если такой пассажир существует в базе данных, и `FALSE` в противном случае. Функция помогает предотвратить дублирование записей и обеспечивает надежность данных о пользователях.

3 Получение количества рейсов по авиакомпании. Для подсчета количества рейсов, связанных с конкретной авиакомпанией, была создана функция `get_flights_by_airline`. Она принимает идентификатор авиакомпании и возвращает количество рейсов, которые принадлежат этой авиакомпании. Это упрощает мониторинг рейсов по каждой авиакомпании и позволяет эффективно анализировать данные.

4 Получение количества багажей по билету. Для подсчета количества багажей, зарегистрированных по определенному билету, была реализована функция `get_baggage_count`. Она принимает идентификатор билета и возвращает количество записей в таблице `baggage`, связанных с данным билетом. Функция помогает отслеживать количество багажа, связанного с каждым билетом, и упрощает управление багажом в системе.

5 Получение списка рейсов с задержками. Для получения информации о рейсах, которые прибывают позже запланированного времени, была создана функция `get_delayed_flights`. Эта функция возвращает таблицу с номерами рейсов, ожидаемыми и фактическими временем прибытия. Это позволяет легко отслеживать задержки рейсов и поддерживать актуальную информацию о расписании.

6 Обновление статуса пользователя при изменении пароля. Для автоматического обновления статуса пользователя в таблице `users` при изменении пароля был создан триггер `user_password_update`. Этот триггер вызывает функцию `update_user_is_active`, которая деактивирует пользователя (устанавливает значение поля `is_active` в `FALSE`) при изменении пароля. Это обеспечивает актуальность данных о пользователях и повышает безопасность

системы.

7 Ограничение на возраст при добавлении паспорта. Для предотвращения создания записи о паспорте для пользователя младше 18 лет был реализован триггер `passport_age_check`. Этот триггер вызывает функцию `check_passport_age`, которая проверяет дату рождения нового пользователя и выбрасывает исключение, если возраст пользователя меньше 18 лет. Это обеспечивает соответствие возрастным ограничениям для выдачи паспорта.

8 Обновление статуса рейса при изменении расписания. Для автоматического обновления статуса рейса в таблице `flight` при изменении времени отправления или прибытия был создан триггер `flight_status_update`. Этот триггер вызывает функцию `update_flight_status`, которая обновляет статус рейса на 'Departed' или 'Arrived' в зависимости от изменения времени реального отправления и прибытия. Это позволяет поддерживать актуальную информацию о статусах рейсов.

9 Уведомление пользователя о новом сообщении. Для отправки уведомления пользователю о новом сообщении была создана функция `notify_user_new_message`. Эта функция автоматически вставляет запись в таблицу `notification`, когда для пользователя создается новое сообщение в таблице `event`. Это позволяет пользователям своевременно получать уведомления о новых сообщениях.

10 Автоматическое добавление рейса при создании маршрута. Для автоматического добавления рейса при создании нового маршрута был разработан триггер `auto_add_flight`. Этот триггер вызывает функцию `add_flight_for_route`, которая создает новый рейс с фиктивными значениями для `plane_id` и `airline_id` при добавлении нового маршрута. Это упрощает процесс добавления рейсов и ускоряет создание новых маршрутов в системе.

Каждый из описанных механизмов интегрирован в архитектуру системы, обеспечивая автоматизацию и снижение вероятности ошибок при работе с данными.

Эти триггеры и процедуры повышают надежность базы данных и упрощают её администрирование.

4.5 Запросы

Этот раздел посвящен описанию ключевых запросов, разработанных для реализации функциональных возможностей системы. Запросы позволяют эффективно извлекать данные из базы для решения различных задач.

Они обеспечивают быстрый доступ к необходимой информации, упрощая работу с базой данных и повышая удобство использования системы.

В проекте реализованы следующие запросы:

1 Информация о пользователях и их национальности. Этот запрос извлекает данные о пользователях, включая их идентификатор, имя, электронную почту и национальность. Используется для получения информации о пользователях в контексте их гражданства.

2 Список всех рейсов с их авиакомпаниями и аэропортами отправления. Запрос возвращает информацию о рейсах, включая номер рейса, имя авиакомпании и название аэропорта отправления. Он удобен для получения общего обзора всех рейсов и их маршрутов.

3 Информация о билетах с местами и классом. Этот запрос извлекает данные о билетах, включая цену, номер места и класс, в котором оно расположено. Он позволяет получить подробности о продаже мест на рейсах.

4 Пассажиры с паспортами и их гражданством. Запрос отображает информацию о пассажирах, включая их имя, фамилию и гражданство, предоставляя данные о паспортах пассажиров и их национальности.

5 Рейсы с информацией о самолётах и аэропортах отправления. Этот запрос предоставляет информацию о рейсах, включая номер рейса, модель самолёта и название аэропорта отправления. Используется для анализа данных о транспортных средствах и местах отправления.

6 Расписание рейсов с информацией о фактических и ожидаемых времени отправления. Запрос извлекает данные о расписаниях рейсов, включая ожидаемое и фактическое время отправления и прибытия, что помогает отслеживать задержки и точность рейсов.

7 Список всех экипажей с их ролями на рейсах. Этот запрос собирает информацию о составе экипажей и их ролях на рейсах. Он полезен для анализа распределения обязанностей среди членов экипажа на различных рейсах.

8 Питание на билетах с их ценой и количеством. Запрос отображает данные о питании, предоставляемом пассажирам, включая наименование блюда, его цену и количество. Он помогает анализировать продажу питания и его ассортимент на рейсах.

9 Оценки пользователей для рейсов. Этот запрос собирает оценки и комментарии пользователей о рейсах, включая имена пользователей, их рейтинги и комментарии. Он позволяет анализировать обратную связь пассажиров и улучшать качество обслуживания. 1

10 Уведомления пользователей с их статусом прочтения. Запрос извлекает информацию о уведомлениях, отправленных пользователям, включая их статус прочтения, что помогает отслеживать эффективность уведомлений и реакцию пользователей на них.

Эти запросы охватывают ключевые аспекты функциональности системы, предоставляя гибкие инструменты для анализа данных и принятия решений. Код запросов представлен в листинге А.4.

5 ТЕСТИРОВАНИЕ РАБОТОСПОСОБНОСТИ БАЗЫ ДАННЫХ

В данном разделе представлены результаты тестирования базы данных, направленные на проверку корректности работы скриптов, созданных для реализации бизнес-логики. Все тесты были выполнены с использованием SQL-запросов, ошибки или успешное выполнение которых зафиксированы на скриншотах.

Тесты на соблюдение ограничений:

1 Проверка уникальности значений в таблице *users*. Для тестирования был выполнен запрос с повторяющимся значением *email*. Запрос завершится с ошибкой из-за нарушения ограничения уникальности (рисунок 5.1).

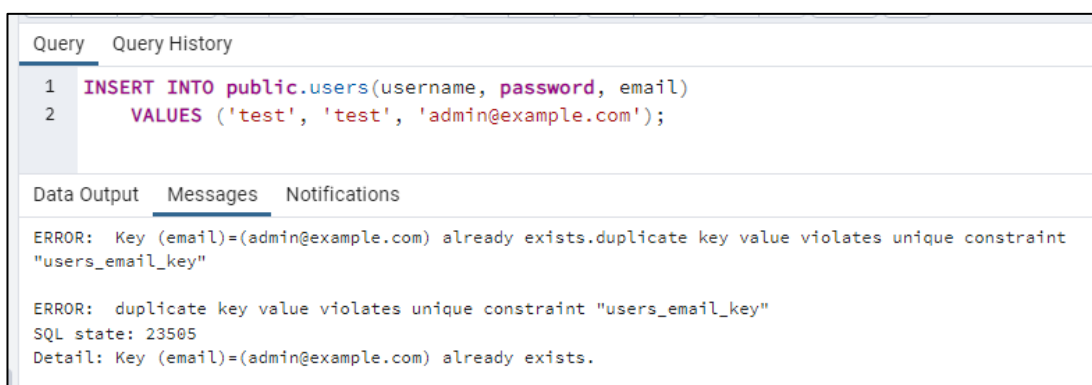


Рисунок 5.1 – Ошибка проверки уникальности

2 Проверка ссылочной целостности. Для тестирования работы внешнего ключа, который ссылается на таблицу *passport*, был выполнен запрос на удаление записи из таблицы *nationality*, на которую ссылается поле *nationality_id* в таблице *passport*. Результат представлен на рисунке 5.2.

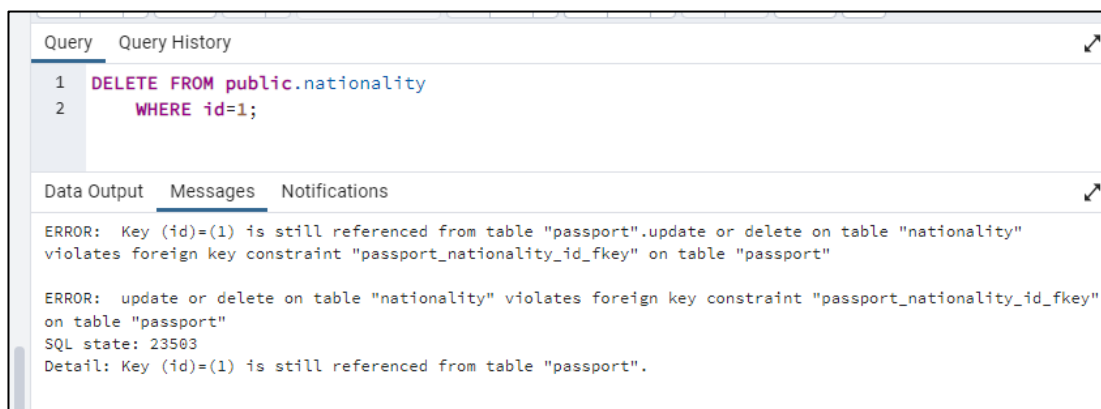


Рисунок 5.2 – Проверка ссылочной целостности

3 Проверка *CHECK* ограничения. Для проверки работы ограничения, запрещающего отрицательные значения для поля *price* таблицы *ticket*, был выполнен запрос, представленный на рисунке 5.3.

Query Query History

1 INSERT INTO public.ticket(reservation_date, price, flight_id, seat_id, passport_id)
2 VALUES ('2024-12-01 10:00:00', -777, 1, 1, 1);

Data Output Messages Notifications

ERROR: Failing row contains (3, 2024-12-01 10:00:00, -777.000, 1, 1, 1).new row for relation "ticket" violates check constraint "ticket_price_check"

ERROR: new row for relation "ticket" violates check constraint "ticket_price_check"

SQL state: 23514

Detail: Failing row contains (3, 2024-12-01 10:00:00, -777.000, 1, 1, 1).

Рисунок 5.3 – Проверка ограничения

Для тестирования триггера на уведомление пользователя о новом сообщении был выполнен запрос на добавление нового *event*. В результате в таблицу *notification* была добавлена строка о новом сообщении для пользователя (рисунок 5.4).

Query Query History

1 -- Тестирования триггера
2 -- 1. Вставляем тестовую запись в таблицу event, чтобы сработал триггер
3 INSERT INTO event (name, datetime, description, user_id, event_type_id)
4 VALUES ('Test Event', NOW(), 'This is a test event for notification trigger', 1, 1);
5
6 -- 2. Проверяем, что в таблице notification появилось новое уведомление
7 SELECT * FROM notification WHERE user_id = 1 AND message = 'You have a new message';

Data Output Messages Notifications

	id [PK] integer	user_id integer	message text	created_at timestamp without time zone	is_read boolean
1	1	1	You have a new message	2024-12-14 20:53:24.901704	false
2	4	1	You have a new message	2024-12-15 12:00:13.630497	false

Рисунок 5.4 – Работа триггера на уведомление пользователя о новом сообщении

ЗАКЛЮЧЕНИЕ

В процессе разработки системы для управления информацией об аэропортах был создан проект базы данных, обеспечивающий централизованное хранение, обработку и предоставление данных о рейсах, аэропортах и авиакомпаниях. Важной частью разработки стало проектирование базы данных с использованием современных технологий и инструментов. Система базы данных была спроектирована с учетом всех требований к целостности, безопасности и производительности данных.

Проектирование базы данных было выполнено в несколько этапов: сначала были созданы инфологическая и даталогическая модели, что позволило точно определить структуру и взаимосвязи данных. После этого была разработана физическая модель базы данных с использованием современных технологий и инструментов. Созданная система обладает возможностью быстрого и эффективного извлечения данных, а также гарантирует минимизацию ошибок и высокую производительность при обработке запросов.

Также был проведен ряд тестов для оценки работоспособности базы данных, в том числе на ограничения, аномалии, а также тесты для процедур и триггеров. Тестирование подтвердило, что система успешно справляется с поставленными задачами, такими как создание, обновление и удаление записей, а также выполнение сложных запросов. Все тесты показали корректность работы системы в различных сценариях, что подтверждает высокую надежность разработанной базы данных.

Результаты данной работы могут найти практическое применение в авиационной отрасли. Созданный агрегатор информации об аэропортах и рейсах предоставляет значительное улучшение в области планирования путешествий и организации воздушных перевозок. Система позволяет оперативно получать информацию о рейсах, авиакомпаниях и аэропортах, что значительно упрощает процесс бронирования и путешествий.

Это повышает эффективность работы аэропортов и авиакомпаний, а также способствует улучшению качества обслуживания пассажиров, что в условиях цифровизации становится важным аспектом для развития отрасли.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Aviasales [Электронный ресурс]. – Режим доступа: <https://www.aviasales.com>. – Дата доступа: 12.10.2024.
- [2] Skyscanner [Электронный ресурс]. – Режим доступа: <https://www.skyscanner.com/>. – Дата доступа: 12.10.2024.
- [3] FlightAware [Электронный ресурс]. – Режим доступа: <https://www.flightaware.com/>. – Дата доступа: 12.10.2024.
- [4] Коннолли, Т. Базы данных : проектирование, реализация и сопровождение. Теория и практика, 2-е изд. / Т. Коннолли, К. Бегг, А. Страчан. – Пер. с англ. — М. : Издательский дом «Вильямс», 2021. – 1120 с.
- [5] Новиков, Б.А. Основы технологий баз данных / Б.А. Новиков, Е. А. Горшкова, Н. Г. Графеева. – Москва : ДМК Пресс, 2020. – 582 с.
- [6] Дюбуа, П. MySQL. Сборник рецептов. / П. Дюбуа, Е. А. Горшкова, Н. Г. Графеева. – Москва : Символ-Плюс, 2017. – 491 с.
- [7] Том, К. Oracle для профессионалов / К. Том, Е. А. Горшкова, Н. Г. Графеева. – СПб. : ООО «ДиаСофтЮП», 2019. – 672 с.
- [8] Руководство по SQLite [Электронный ресурс]. – Режим доступа: <https://metanit.com/sql/sqlite/>. – Дата доступа: 28.11.2024.
- [9] Стружкин, Н. П. Базы данных: проектирование. Практикум : учебное пособие для вузов / Н. П. Стружкин, В. В. Годин. — Москва : Издательство Юрайт, 2024. — 291 с.
- [10] Осипов Д. Технологии проектирования баз данных. – М.: ДМК Пресс, 2019. – 498 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг А.1 – Файл *create-db.sql*

```
DROP SCHEMA public CASCADE;
CREATE SCHEMA public;
GRANT ALL ON SCHEMA public TO public;

-- Table "user"
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(45) NOT NULL UNIQUE,
    password VARCHAR(64) NOT NULL,
    email VARCHAR(254) NOT NULL UNIQUE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    is_admin BOOLEAN NOT NULL DEFAULT FALSE,
    is_staff BOOLEAN NOT NULL DEFAULT FALSE,
    CHECK (LENGTH(username) > 0),
    CHECK (LENGTH(password) > 0)
);

-- Table nationality
CREATE TABLE nationality (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL UNIQUE,
    CHECK (LENGTH(name) > 0)
);

-- Table gender
CREATE TABLE gender (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL UNIQUE,
    CHECK (LENGTH(name) > 0)
);

-- Table country
CREATE TABLE country (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL UNIQUE,
    code VARCHAR(3) NOT NULL UNIQUE,
    CHECK (LENGTH(name) > 0),
    CHECK (LENGTH(code) > 0)
);

-- Table passport
CREATE TABLE passport (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(45) NOT NULL,
    last_name VARCHAR(45) NOT NULL,
    birth_date DATE NOT NULL,
    pass_number VARCHAR(45) NOT NULL,
    identification_number VARCHAR(45) NOT NULL,
    issue_date DATE NOT NULL,
    expiry_date DATE NOT NULL,
    nationality_id INT NOT NULL REFERENCES nationality(id),
    gender_id INT NOT NULL REFERENCES gender(id),
    country_id INT NOT NULL REFERENCES country(id),
```

```

        user_id INT REFERENCES users(id) ON DELETE SET NULL,
        CHECK (expiry_date > issue_date)
    );

-- Table city
CREATE TABLE city (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL,
    country_id INT NOT NULL REFERENCES country(id),
    CHECK (LENGTH(name) > 0)
);

-- Table airport
CREATE TABLE airport (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    code_IATA VARCHAR(3) NOT NULL,
    city_id INT NOT NULL UNIQUE REFERENCES city(id),
    CHECK (LENGTH(name) > 0),
    CHECK (LENGTH(code_IATA) > 0)
);

-- Table plane
CREATE TABLE plane (
    id SERIAL PRIMARY KEY,
    model VARCHAR(45) NOT NULL,
    capacity INT NOT NULL,
    registration_number VARCHAR(20) NOT NULL UNIQUE,
    is_under_maintenance BOOLEAN NOT NULL,
    CHECK (capacity > 0)
);

-- Table airline
CREATE TABLE airline (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL,
    description TEXT,
    country_id INT NOT NULL REFERENCES country(id),
    CHECK (LENGTH(name) > 0)
);

-- Table flight
CREATE TABLE flight (
    id SERIAL PRIMARY KEY,
    flight_number VARCHAR(10) NOT NULL,
    airport_id INT NOT NULL REFERENCES airport(id),
    plane_id INT NOT NULL REFERENCES plane(id),
    airline_id INT NOT NULL REFERENCES airline(id),
    CHECK (LENGTH(flight_number) > 0)
);

-- Table seatclass
CREATE TABLE seatclass (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL CHECK (LENGTH(name) > 0)
);

-- Table seat
CREATE TABLE seat (
    id SERIAL PRIMARY KEY,
    number VARCHAR(1) NOT NULL,
    row_num INT NOT NULL,

```

```

        seatclass_id INT NOT NULL REFERENCES seatclass(id),
        plane_id INT NOT NULL REFERENCES plane(id) ON DELETE CASCADE
    );

-- Table ticket
CREATE TABLE ticket (
    id SERIAL PRIMARY KEY,
    reservation_date TIMESTAMP NOT NULL,
    price DECIMAL(10,3) NOT NULL,
    flight_id INT NOT NULL REFERENCES flight(id),
    seat_id INT NOT NULL REFERENCES seat(id),
    passport_id INT NOT NULL REFERENCES passport(id),
    CHECK (price >= 0)
);

-- Table schedule
CREATE TABLE schedule (
    id SERIAL PRIMARY KEY,
    expected_departure TIMESTAMP NOT NULL,
    expected_arrival TIMESTAMP NOT NULL,
    real_departure TIMESTAMP,
    real_arrival TIMESTAMP,
    flight_id INT NOT NULL UNIQUE REFERENCES flight(id) ON DELETE CASCADE,
    CHECK (expected_departure < expected_arrival),
    CHECK (real_departure IS NULL OR real_departure < real_arrival)
);

-- Table baggage
CREATE TABLE baggage (
    id SERIAL PRIMARY KEY,
    weight DECIMAL(5,3) NOT NULL,
    ticket_id INT REFERENCES ticket(id) ON DELETE SET NULL,
    CHECK (weight >= 0)
);

-- Table event_type
CREATE TABLE event_type (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL UNIQUE CHECK (LENGTH(name) > 0)
);

-- Table event
CREATE TABLE event (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL,
    datetime TIMESTAMP NOT NULL,
    description TEXT,
    user_id INT REFERENCES users(id) ON DELETE SET NULL,
    event_type_id INT NOT NULL REFERENCES event_type(id),
    CHECK (LENGTH(name) > 0)
);

-- Table crew
CREATE TABLE crew (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(45) NOT NULL,
    last_name VARCHAR(45) NOT NULL,
    position VARCHAR(45) NOT NULL,
    nationality_id INT REFERENCES nationality(id),
    CHECK (LENGTH(first_name) > 0),
    CHECK (LENGTH(last_name) > 0)
);

```

```

-- Table crew_assignment
CREATE TABLE crew_assignment (
    id SERIAL PRIMARY KEY,
    crew_id INT NOT NULL REFERENCES crew(id) ON DELETE CASCADE,
    flight_id INT NOT NULL REFERENCES flight(id) ON DELETE CASCADE,
    role VARCHAR(45) NOT NULL
);

-- Table route
CREATE TABLE route (
    id SERIAL PRIMARY KEY,
    source_airport_id INT NOT NULL REFERENCES airport(id),
    destination_airport_id INT NOT NULL REFERENCES airport(id),
    distance_km DECIMAL(8,2) NOT NULL CHECK (distance_km > 0)
);

-- Table meal
CREATE TABLE meal (
    id SERIAL PRIMARY KEY,
    name VARCHAR(45) NOT NULL UNIQUE,
    description TEXT,
    price DECIMAL(5,2) NOT NULL CHECK (price >= 0)
);

-- Table ticket_meal
CREATE TABLE ticket_meal (
    id SERIAL PRIMARY KEY,
    ticket_id INT NOT NULL REFERENCES ticket(id) ON DELETE CASCADE,
    meal_id INT NOT NULL REFERENCES meal(id) ON DELETE CASCADE,
    quantity INT NOT NULL DEFAULT 1 CHECK (quantity > 0)
);

-- Table payment
CREATE TABLE payment (
    id SERIAL PRIMARY KEY,
    ticket_id INT NOT NULL REFERENCES ticket(id),
    payment_date TIMESTAMP NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(45) NOT NULL,
    CHECK (amount >= 0)
);

-- Table review
CREATE TABLE review (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id),
    flight_id INT NOT NULL REFERENCES flight(id),
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comments TEXT
);

-- Table notification
CREATE TABLE notification (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id),
    message TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_read BOOLEAN DEFAULT FALSE
);

```

Листинг А.2 – Файл *indexes.sql*

```
-- Индексы для таблицы user
CREATE INDEX IF NOT EXISTS idx_user_username ON "user"(username);
CREATE INDEX IF NOT EXISTS idx_user_email ON "user"(email);

-- Индексы для таблицы passport
CREATE INDEX IF NOT EXISTS idx_passport_user_id ON passport(user_id);
CREATE INDEX IF NOT EXISTS idx_passport_nationality_id ON
passport(nationality_id);
CREATE INDEX IF NOT EXISTS idx_passport_country_id ON passport(country_id);
CREATE INDEX IF NOT EXISTS idx_passport_gender_id ON passport(gender_id);

-- Индексы для таблицы country
CREATE INDEX IF NOT EXISTS idx_country_code ON country(code);

-- Индексы для таблицы city
CREATE INDEX IF NOT EXISTS idx_city_country_id ON city(country_id);

-- Индексы для таблицы airport
CREATE INDEX IF NOT EXISTS idx_airport_city_id ON airport(city_id);
CREATE INDEX IF NOT EXISTS idx_airport_code_IATA ON airport(code_IATA);

-- Индексы для таблицы plane
CREATE INDEX IF NOT EXISTS idx_plane_registration_number ON
plane(registration_number);

-- Индексы для таблицы airline
CREATE INDEX IF NOT EXISTS idx_airline_country_id ON airline(country_id);

-- Индексы для таблицы flight
CREATE INDEX IF NOT EXISTS idx_flight_airport_id ON flight(airport_id);
CREATE INDEX IF NOT EXISTS idx_flight_plane_id ON flight(plane_id);
CREATE INDEX IF NOT EXISTS idx_flight_airline_id ON flight(airline_id);
CREATE INDEX IF NOT EXISTS idx_flight_flight_number ON flight(flight_number);

-- Индексы для таблицы seat
CREATE INDEX IF NOT EXISTS idx_seat_plane_id ON seat(plane_id);
CREATE INDEX IF NOT EXISTS idx_seat_seatclass_id ON seat(seatclass_id);

-- Индексы для таблицы ticket
CREATE INDEX IF NOT EXISTS idx_ticket_flight_id ON ticket(flight_id);
CREATE INDEX IF NOT EXISTS idx_ticket_seat_id ON ticket(seat_id);
CREATE INDEX IF NOT EXISTS idx_ticket_passport_id ON ticket(passport_id);

-- Индексы для таблицы schedule
CREATE INDEX IF NOT EXISTS idx_schedule_flight_id ON schedule(flight_id);

-- Индексы для таблицы baggage
CREATE INDEX IF NOT EXISTS idx_baggage_ticket_id ON baggage(ticket_id);

-- Индексы для таблицы event
CREATE INDEX IF NOT EXISTS idx_event_user_id ON event(user_id);
CREATE INDEX IF NOT EXISTS idx_event_event_type_id ON event(event_type_id);

-- Индексы для таблицы plane_maintenance
CREATE INDEX IF NOT EXISTS idx_plane_maintenance_plane_id ON
plane_maintenance(plane_id);

-- Индексы для таблицы crew
CREATE INDEX IF NOT EXISTS idx_crew_nationality_id ON crew(nationality_id);
```

```

-- Индексы для таблицы crew_assignment
CREATE INDEX IF NOT EXISTS idx_crew_assignment_crew_id ON
crew_assignment(crew_id);
CREATE INDEX IF NOT EXISTS idx_crew_assignment_flight_id ON
crew_assignment(flight_id);

-- Индексы для таблицы route
CREATE INDEX IF NOT EXISTS idx_route_source_airport_id ON
route(source_airport_id);
CREATE INDEX IF NOT EXISTS idx_route_destination_airport_id ON
route(destination_airport_id);

-- Индексы для таблицы ticket_meal
CREATE INDEX IF NOT EXISTS idx_ticket_meal_ticket_id ON
ticket_meal(ticket_id);
CREATE INDEX IF NOT EXISTS idx_ticket_meal_meal_id ON ticket_meal(meal_id);

-- Индексы для таблицы payment
CREATE INDEX IF NOT EXISTS idx_payment_ticket_id ON payment(ticket_id);

-- Индексы для таблицы review
CREATE INDEX IF NOT EXISTS idx_review_user_id ON review(user_id);
CREATE INDEX IF NOT EXISTS idx_review_flight_id ON review(flight_id);

-- Индексы для таблицы notification
CREATE INDEX IF NOT EXISTS idx_notification_user_id ON notification(user_id);
CREATE INDEX IF NOT EXISTS idx_notification_is_read ON notification(is_read);

-- Индексы для таблицы service
CREATE INDEX IF NOT EXISTS idx_service_name ON service(name);

```

Листинг А.3 – Файл *triggers-functions.sql*

```

-- Функция 1: Получить количество активных пользователей
CREATE OR REPLACE FUNCTION get_active_users_count() RETURNS INT AS $$
BEGIN
    RETURN (SELECT COUNT(*) FROM users WHERE is_active = TRUE);
END;
$$ LANGUAGE plpgsql;

-- Функция 2: Проверить, есть ли пассажир с таким паспортом
CREATE OR REPLACE FUNCTION check_passport_exists(passport_number VARCHAR)
RETURNS BOOLEAN AS $$
BEGIN
    RETURN EXISTS (SELECT 1 FROM passport WHERE pass_number =
passport_number);
END;
$$ LANGUAGE plpgsql;

-- Функция 3: Получить количество рейсов по авиакомпании
CREATE OR REPLACE FUNCTION get_flights_by_airline(airline_id INT) RETURNS INT
AS $$
BEGIN
    RETURN (SELECT COUNT(*) FROM flight WHERE airline_id = airline_id);
END;
$$ LANGUAGE plpgsql;

-- Функция 4: Получить количество багажей по билету
CREATE OR REPLACE FUNCTION get_baggage_count(ticket_id INT) RETURNS INT AS $$
BEGIN
    RETURN (SELECT COUNT(*) FROM baggage WHERE ticket_id = ticket_id);
END;

```



```

$$ LANGUAGE plpgsql;

-- Функция 5: Получить список рейсов с задержками
CREATE OR REPLACE FUNCTION get_delayed_flights() RETURNS TABLE(flight_number
VARCHAR, expected_arrival TIMESTAMP, real_arrival TIMESTAMP) AS $$
BEGIN
    RETURN QUERY
    SELECT f.flight_number, s.expected_arrival, s.real_arrival
    FROM flight f
    JOIN schedule s ON f.id = s.flight_id
    WHERE s.real_arrival > s.expected_arrival;
END;
$$ LANGUAGE plpgsql;

-- Триггер 1: Автоматическое обновление поля "is_active" при изменении пароля
пользователя
CREATE OR REPLACE FUNCTION update_user_is_active() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.password <> OLD.password THEN
        UPDATE users SET is_active = FALSE WHERE id = NEW.id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER user_password_update
AFTER UPDATE OF password
ON users
FOR EACH ROW
EXECUTE FUNCTION update_user_is_active();

-- Триггер 2: Ограничение на возраст при добавлении паспорта
CREATE OR REPLACE FUNCTION check_passport_age() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.birth_date > CURRENT_DATE - INTERVAL '18 years' THEN
        RAISE EXCEPTION 'Passport can only be issued for users over 18 years
old';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER passport_age_check
BEFORE INSERT OR UPDATE ON passport
FOR EACH ROW
EXECUTE FUNCTION check_passport_age();

-- Триггер 3: Обновление статуса рейса при изменении расписания
CREATE OR REPLACE FUNCTION update_flight_status() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.real_departure IS NOT NULL THEN
        UPDATE flight SET status = 'Departed' WHERE id = NEW.flight_id;
    END IF;
    IF NEW.real_arrival IS NOT NULL THEN
        UPDATE flight SET status = 'Arrived' WHERE id = NEW.flight_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER flight_status_update
AFTER UPDATE OF real_departure, real_arrival

```

```

ON schedule
FOR EACH ROW
EXECUTE FUNCTION update_flight_status();

-- Триггер 4: Уведомление пользователя о новом сообщении
CREATE OR REPLACE FUNCTION notify_user_new_message() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO notification (user_id, message) VALUES (NEW.user_id, 'You
have a new message');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER new_message_notification
AFTER INSERT ON event
FOR EACH ROW
EXECUTE FUNCTION notify_user_new_message();

-- Триггер 5: Автоматическое добавление рейса при создании маршрута
CREATE OR REPLACE FUNCTION add_flight_for_route() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO flight (flight_number, airport_id, plane_id, airline_id)
    VALUES (CONCAT('F', NEW.id), NEW.source_airport_id, 1, 1); -- Пример с
использованием фиктивных значений для plane_id и airline_id
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER auto_add_flight
AFTER INSERT ON route
FOR EACH ROW
EXECUTE FUNCTION add_flight_for_route();

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема базы данных

ПРИЛОЖЕНИЕ В
(обязательное)
Ведомость документов